# Trabajo Práctico Integrador - Programación II

**Food Store - Sistema de Gestión de Pedidos de Comida**

**Institución:** Universidad Tecnológica Nacional - Facultad Regional Mendoza (UTN FRM)

**Carrera:** Tecnicatura Universitaria en Programación

**Materia:** Programación II

**Estudiantes:** Italo Rocamora | Joaquín Martinez

**Fecha de entrega:** 9 de Noviembre 2026

# ÍNDICE

**1. Marco teórico**

**2. Decisiones técnicas y arquitectura**

**3. Dificultades y soluciones**

**4. Bibliografía**

**5. Enlaces**

# 1. Marco Teórico

En este proyecto se usaron y aplicaron los pilares de la Programación Orientada a Objetos (POO) aprendidos en el cursado de la materia:

- **Abstracción y Herencia:** Se diseñó la superclase abstracta Base, de la cual heredan todas las entidades del dominio (Categoria, Producto, Usuario, Pedido, DetallePedido). Esto permite centralizar atributos comunes como el ID , el estado de eliminación lógica (eliminado) y la fecha de creación (creado_en).
  Se utilizó el módulo abc de Python el cual permite la abstracción.
- **Interfaz y Polimorfismo (Modelo):** Se implementó la clase abstracta pura que se llama “Calculable”, ella define el contrato (método calcular_total()) que la clase Pedido debe usar. De esa manera se “firma” el contrato permitiendo un comportamiento predecible y estándar.
- **Polimorfismo (Consola):** Para mostrar los menús según el rol del usuario (Administrador vs. Cliente), se usó una clase abstracta Menu y sus subclases (MenuAdministrador y MenuCliente).
  La lógica principal solo interactúa con la interfaz común (método mostrar()), lo cual evita estar usando condicionales grandes if/else.
- **Encapsulamiento:** Se protegió el estado interno de todos los objetos usando atributos privados (__(doble guión abjo)). El acceso y modificación se logra mediante decoradores @property y setters con validaciones (ej. impidiendo la asignación de stocks o precios negativos en Producto).
- **Composición y Agregación:** Se usó la agregación entre Producto y Categoria, y una composición entre Pedido y DetallePedido, lo cual nos permite estar seguros que la vida de los detalles depende del todo “Pedido”.
- **Type Hints:** El código fuente fue documentado usando anotaciones de tipo (Type Hints) para atributos, parámetros y retornos, aumentando la robustez y legibilidad del código.
- **Excepciones Personalizadas:** Se logró el bloque de excepciones propias que el TPI solicitaba (heredan de Exception) como FoodStoreError, StockInsuficienteError, y CredencialesInvalidasError.
  Esto permite separar los errores lógicos del negocio de las fallas del sistema, lo que asegura y nos hace quedar tranquilos de que la consola nunca se cierre de forma inesperada. Es decir separamos lo horror que se puedan dar en el Dominio del TPI de los errores de programación.

# 2. Decisiones Técnicas y Arquitectura

Diseñamos el sistema respetando la arquitectura de tres capas, teniendo una separación de responsabilidades:

- **Entidades (food_store/entidades/):** Clases del dominio que encapsulan su propio estado y reglas básicas ( datos validados  en setters). No interactúan con la interfaz de usuario.

- **Servicios (food_store/servicios/):** Esta es la capa que centraliza la lógica de negocio compleja, controla las listas en memoria y lanza las excepciones de dominio que programamos. Es como un intermediario entre los datos y la consola.
- **Consola (food_store/consola/):** Es la capa de presentación que se encarga de la interacción con el usuario gracias a las entradas y salidas. Atrapa las excepciones de la capa de servicios y las convierte en mensajes claros, lo cual mejora la experiencia del usuario, ya sea Admin o Usuario (común).
- **Almacenamiento:** Se respeta el  uso de colecciones en memoria (listas a nivel de módulo) para que los datos sólo persisten durante la ejecución, usando datos de prueba iniciales para facilitar la evaluación.

- **"Prueba.py":** Se creo este archivo ya que era solictio, con este probamos que el código se ejecutara correctamente cumpliendo con las validaciones y demás, es codigo corto pero que en su momento nos permitia ver si algo fallaba para poder ir modificando el código hasta que "pruebas.py" funcionara correctamente.

- **Decisiones Técnicas y Arquitectura**:
  Para la generación de identificadores, se optó por implementar un único contador de clase (_id_counter) compartido dentro de la superclase abstracta Base. Esta decisión garantiza que cada entidad instanciada posea un ID único. Además, este enfoque evita que dupliquemos código, evitando tener que declarar contadores individuales en cada una de las subclases del dominio del codigo,

- **Implementación de pruebas automáticas(pruebas.py):**
  Para el archivo de validación del sistema, se eligió usar assert de Python en lugar de incorporar librerías no estándares de testing. Esta asegura el cumplimientodel alcance del proyecto, el cual pide el uso de la biblioteca estándar de Python. La instrucción assert nos deja aplicar un enfoque de validación falla rápida: el script evalúa el estado interno de las entidades despues de hacer operaciones que pueden modificar su estado y, si el resultado no es el que se espera, frena la ejecución lanzando un AssertionError. Esto nos da un mecanismo de control eficiente y no dependemos de librerías externas.

## 3. Dificultades y Soluciones

Durante el desarrollo del proyecto, tuvimos y resolvimos los siguientes desafíos:

- **Transaccionalidad (El enfoque "Todo o Nada" con el stock):**
  La dificultad: Si en un mismo pedido un cliente solicita 3 productos diferentes, y el tercero falla porque no hay stock, el sistema no debería descontar del inventario los 2 primeros productos (porque sino el pedido completo se cancela).
  Solución: En el archivo menu_administrador.py, creamos el objeto Pedido temporal en memoria y le fuimos agregando detalles usando pedido.agregar_detalle(...). Si en salta un error, el proceso se interrumpe y el pedido se descarta. Solo si el usuario termina de cargar todo sin errores, se llama a self._ped_srv.registrar(pedido), y es ese método el que hace la resta real del stock.

- **Devolución de stock en bajas lógicas y cancelaciones:**
  Dificultad: seguir el historial de cambios de los pedidos (baja lógica) necesitaba que el stock fuera reintegrado al inventario sin perder el registro del pedido.
  Solución: La lógica de cancelación y eliminación en PedidoServicio incluye un bucle inverso que recorre los detalles del pedido afectado (si no estaba en estado TERMINADO o CANCELADO previamente) y suma nuevamente las cantidades a los productos correspondientes antes de dejar el estado final definitivo.
- **Prevención de cierres inesperados en la captura de datos:**
  Dificultad: Evitar excepciones de tipo ValueError que cerraran el programa cuando el usuario ingresaba textos en lugar de números o dejaba campos obligatorios vacíos.
  Solución: Se desarrolló un módulo de utilidades (lectura.py) que envuelve las funciones input() en bucles try-except. Estos bucles no se rompen hasta que el ingreso sea válido según el tipo de dato esperado, garantizando la estabilidad de los menús.
- **Importaciones:**
  Dificultad:Cuando el código empezó a crecer tuvimos problemas con las importaciones, python no encontraba los módulos y nos daba errores de ModuleNotFoundError.
  La solución: forzar al sistema a reconocer la raíz del proyecto para que las importaciones funcionaran correctamente.

- **Importaciones Circulares:**
  Dificultad: Al cumplir con el requerimiento de documentar todo el código con Type Hints, nos encontramos con errores de importaciones circulares. Por ejemplo, al intentar referenciar entidades entre sí dentro de los módulos, el intérprete entraba en un bucle de dependencias durante el tiempo de ejecución, provocando la caida del código en sí.
  Solución: utilizamos from __future__ import annotations al inicio de los archivos y el bloque condicional if TYPE_CHECKING: del módulo typing. Esto nos permitió ocultar las importaciones cruzadas al intérprete de Python durante la ejecución real del programa. Así cumplimos con el type hints requerido sin comprometer la estabilidad y el inicio del sistema.

# 4. Capturas del código:

### a. Menús por rol:

#### i. Admin



ii. **Usuario:**

```
Sesión cerrada. Volviendo al inicio de sesión...

=== FOOD STORE – INICIO DE SESIÓN ===
Mail: juan@mail.com
Contraseña: juan123

Bienvenido/a, Juan Gómez [USUARIO]

=== FOOD STORE – CLIENTE (Juan Gómez) ===
1. Ver categorías
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesión
0. Salir
Seleccione:
```

**b. Todo o Nada (Transaccionalidad):**

```
=== FOOD STORE – ADMINISTRADOR (Ana Pérez) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
9. Cerrar sesión
0. Salir
Seleccione: 4

--- PEDIDOS ---
1. Listar
2. Crear
3. Cambiar estado / forma de pago
4. Eliminar
5. Ver detalles de un pedido
0. Volver
Seleccione: 2
Usuarios disponibles:
[1] Ana Pérez | admin@foodstore.com | 2611234567 | ADMIN
[10] Juan Gómez | juan@mail.com | 2614567890 | USUARIO
```

```
ID del usuario para el pedido: 1
Forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione: 1

-- Agregar productos al pedido (ingrese 0 para terminar) --
  [5] Burger Clásica | $3500.00 | Stock: 20 | Disponible | Cat: Hamburguesas
  [6] Burger Veggie | $3200.00 | Stock: 15 | Disponible | Cat: Hamburguesas
  [7] Coca-Cola 500ml | $1500.00 | Stock: 50 | Disponible | Cat: Bebidas
  [8] Agua Mineral 500ml | $1000.00 | Stock: 40 | Disponible | Cat: Bebidas
  [9] Brownie | $2000.00 | Stock: 10 | Disponible | Cat: Postres

ID del producto (0 para finalizar): 5
Cantidad: 1
  Agregado: Burger Clásica x1

ID del producto (0 para finalizar): 5
Cantidad: 999
Error: Stock insuficiente para 'Burger Clásica'. Disponible: 20, pedido: 999.
```

**c. Consola-Aislamiento ante errores de tipeo:**

```
=== FOOD STORE – ADMINISTRADOR (Ana Pérez) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
9. Cerrar sesión
0. Salir
Seleccione: 2

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Papas
Descripción: Papas con cheddar
Precio: palabra
Error de entrada: could not convert string to float: 'palabra'
```

**d. Baja lógica:**

```
--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
[2] Hamburguesas - Variedad de hamburguesas artesanales
[3] Bebidas - Gaseosas, jugos y agua
[4] Postres - Dulces y helados

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Prueba
Descripción: pruebas
Categoría creada con id 13.
```

```
--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
[2] Hamburguesas - Variedad de hamburguesas artesanales
[3] Bebidas - Gaseosas, jugos y agua
[4] Postres - Dulces y helados
[13] Prueba - pruebas
```

```
[2] Hamburguesas - Variedad de hamburguesas artesanales
[3] Bebidas - Gaseosas, jugos y agua
[4] Postres - Dulces y helados
[13] Prueba - pruebas

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 4
[2] Hamburguesas - Variedad de hamburguesas artesanales
[3] Bebidas - Gaseosas, jugos y agua
[4] Postres - Dulces y helados
[13] Prueba - pruebas
ID de la categoría a eliminar: 13
¿Confirmar? (S/N): s
Categoría eliminada.
```

## 5. Bibliografía


- Documentación oficial de Python: https://docs.python.org/3/
- Apuntes y material teórico de la cátedra de Programación II, Tecnicatura Universitaria en Programación (TUP) - UTN FRM.
- Videos de la cátedra.


## 6. Enlaces

- **Repositorio GitHub: GitHub - Italius-Stoneberry/UTN-TUPaD-Programacion2: Repositorio creado para el trabajo integrador de Programacion 2 · GitHub**
- **Video Demostración: todavía no lo realizamos profesores, vamos a esperar la corrección.**